

CASE STUDY

Conflict Resolution Simulator

AI-powered role-play for workplace soft skills training

Raquel Miranda · Learning Experience Designer · conflict-simulator.netlify.app

Overview

Conflict Resolution Simulator is a browser-based role-play tool powered by Claude. Users have a live text conversation with one of four AI characters, each sitting in a specific workplace conflict. The characters respond to how the user communicates: empathy opens them up, blame makes them harder to reach. After the session, an AI coach analyses the conversation and gives structured feedback on the communication techniques the user actually used.

Designed to work as a standalone training module or as part of a larger program.

Why this exists

Conflict at work is the thing most professionals say they dread and least feel equipped to handle. And the training that exists for it is, almost universally, passive: watch a video, read a scenario, discuss with a group. You never actually have the conversation.

Knowing how non-violent communication works and applying it when someone is being passive-aggressive are completely different skills. The gap between them does not close by watching someone else do it. It closes by doing it yourself, badly, a few times, in a place where the stakes are low enough to survive the learning curve.

That is what this simulator is for.

What I built

Four scenarios, each with a character who has a real emotional logic:

- Lucas, a junior colleague who missed a critical deadline and caused a production bug
- Rita, a senior colleague who presented your idea to leadership as her own
- Daniel, your manager who changed the project requirements for the third time this week
- Marina, a client furious about a deliverable that missed her unannounced repositioning

Each character runs in two modes. Normal has no message limit so you can explore, try different approaches, find your rhythm. Challenge caps the conversation at 10 messages. Enough to feel the pressure of a situation that needs to go somewhere, not enough to meander.

At the end of each session, the user gets a performance badge, an outcome indicator, a radar chart across four skill dimensions, and a detailed analysis that references specific moments in the conversation, pointing to what the user actually said.

How I thought about it

I chose conflict resolution as the theme because it trains everything at once: empathy, assertiveness, active listening, de-escalation, non-violent communication. If you can navigate a genuinely difficult conversation, most other soft skills follow.

The hardest part was writing the characters. Early versions were difficult in a flat, mechanical way: they said defensive things without feeling defensive. What fixed it was taking each character's emotional logic seriously. Rita genuinely believes she added enough value to the idea that the question of credit is complicated, for example. That belief makes her consistent in a way generic "difficult colleague" instructions never produced. I found myself intimidated by Rita in the hardest mode, even knowing exactly what she was programmed to do. I second-guessed whether my frustration was legitimate. That is gaslighting working as intended, and it working on me confirmed the fidelity was right.

I also learned that the disrespect rule, which ends the conversation and triggers a narrative consequence if the user crosses into verbal abuse, is not just a guardrail. It is a design choice about what the simulator is actually teaching. Composure

under pressure is a skill. Making the cost of losing it visible, even in a practice environment, is part of the lesson.

One technical decision I got wrong and fixed: early feedback used a single prompt that tried to detect the conversation language. It kept mixing languages when conversations were short. The obvious solution was two separate prompts, one per language, selected by interface state. I should have started there.

Technical notes

Vanilla JavaScript, no framework. I made that choice deliberately: no build step, no dependencies to break, nothing between the code and anyone who wants to understand it. If you open the repository, you can read what is happening. I think that matters, especially for a portfolio piece.

Two models for two different jobs. Claude Haiku handles the role-play because conversational speed matters. Claude Sonnet handles the feedback because that task needs more careful reasoning. The feedback prompt returns structured JSON alongside prose, which the application uses to render the visual layer separately from the text analysis.

The current version makes direct browser-to-API calls, which works fine for a prototype. Production would route through a backend proxy. That decision is documented in the README, because I think being honest about what is and is not production-ready is part of doing the work well.

What comes next

Simulators are the best format for soft skills training because they close the gap between knowing and doing. This is one module, I am building more: client management, giving difficult feedback upward, navigating a performance conversation. Same architecture, new characters, new contexts.

I am also thinking about onboarding. New employees could practice the conversations they will actually have in their first 90 days before those conversations have professional consequences. The use case is obvious. The simulator architecture already supports it.

Raquel Miranda · [linkedin.com/in/raquelgmiranda](https://www.linkedin.com/in/raquelgmiranda) · conflict-simulator.netlify.app